УНИВЕРЗИТЕТ У БЕОГРАДУ
ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ

Катедра за Електронику

Предмет: Дигитални VLSI системи

Пројекат:
Убрзање филтрирања слике

Ментор:
Доц. др Драгомир Ел Мезени

Студенти:
Михајло Павловић 21/0343
Јован Јеремић 16/0554

Март, 2026.

1. Увод **3**
2. Реализација хардверског дела система **3**
2.1. Архитектура хардверског акцелератора 4
2.2.1. BRAM меморијски подсистем 5
2.2.2. Јединица за обраду пиксела 5
2.2.3. Регистарска компонента 7
2.2. Синтеза и имплементација система 8
3. Реализација софтверског дела система **9**
3.1. Структура 9
3.2. Опис функција 10
3.3. Код функције ImageFilterSW 12
4. Резултати и мерења **15**
5. Закључак **16**

1. Увод

За потребе предмета *Дигитални VLSI системи* (ДВС), имплементирали смо функције линеарног, просторно инваријантног филтрирања слике. Од нас се захтевало да извршимо софтверско филтрирање као и хардверско. На располагању смо имали *PYNQ-Z2* SOC плочу серије *ZYNQ 7000*, Xilinx произвођача, са ARM *Cortex-A9 FPGA* процесорским чипом. Израда хардвера рађена је у Vivado програмском алату у VHDL програмском језику, а израда софтвера рађена је у Vitis програмском окружењу у C програмском језику, оба такође од произвођача Xilinx.

Било је потребно имплементирати акцелератор који би кроз 128x128 или 512x512 слике пропустио box, Гаусов или LOG филтер, кернел величине 3x3, 5x5, 7x7 или 9x9. Пошто би корелација слике са кернелом смањила димензије слике, акцелератор има имплементирану могућност привидног проширивања слике како не би дошло до децимације. Било је потребно реализовати опције без проширења, проширења са константним вредностима и проширења са вредностима ивице. Како би поједини филтери правилно функционисали, требало је реализовати и функционалност 8-битног означеног излаза као и 16-битног означеног излаза са разломљеним делом.

Акцелератор такође мора да има реализовану функционалност чувања конфигурационих вредности унутар интерних регистра, који могу бити приступљени током извршавања програма, али не током генерисања излазне слике. Остатак система је у оквиру себе садржао DMA, AXI Timer, Z2 процесор, којима је убрзавао и комплетирао његову основну функционалност и опонашао реалан систем.

2. Реализација хардверског дела система

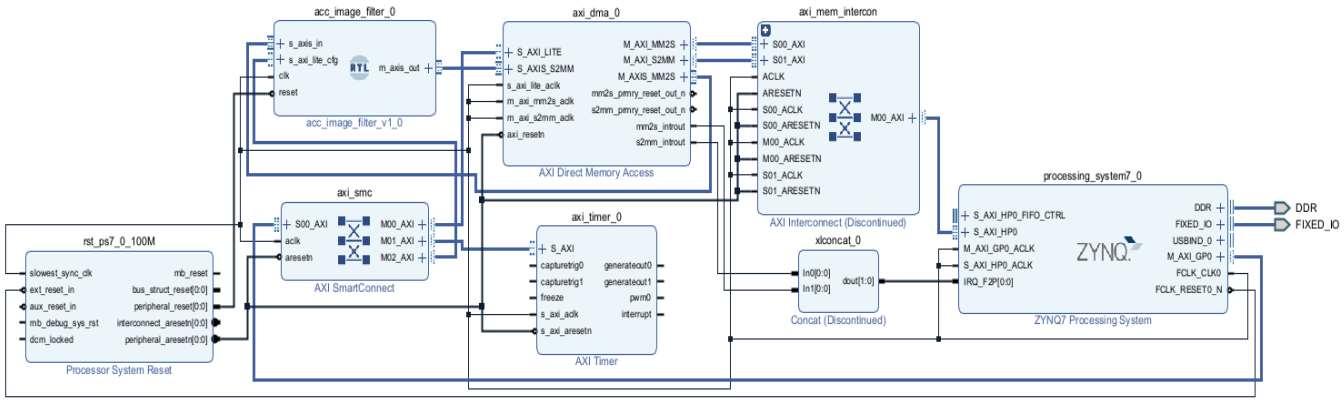
Систем има намену да убрза филтрирање дигиталне слике коришћењем наменски пројектованог хардверског акцелератора, модификовањем *AXI Lite* модула. Основни циљ пројектовања био је постизање значајног убрзања у односу на софтверску имплементацију филтрирања, уз задржавање флексибилности и могућности конфигурације параметара у *run-time* режиму рада.

Конфигурациони параметри задају се у оквиру *Vitis* платформе. Систем подржава обраду слике произвољних димензија до максималне величине дефинисане расположивом меморијом. Параметри филтрирања, димензије слике и режим рада акцелератора конфигуришу се преко регистарског интерфејса.

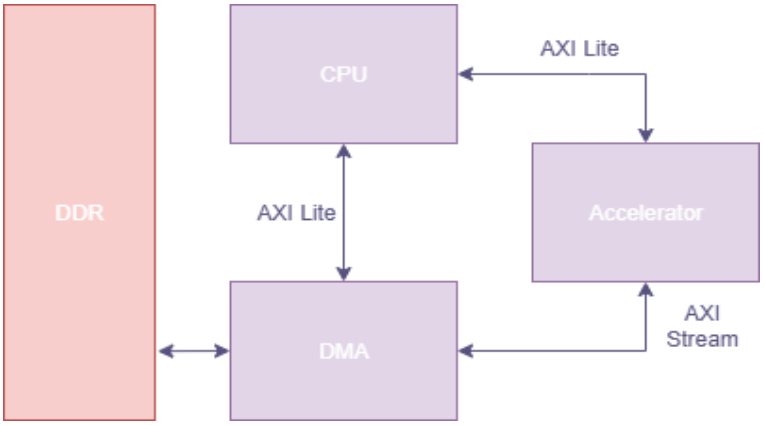
Задате параметре рада као и коефицијенте филтера, модификовани *AXI lite* модул чита и уписује у интерне регистре. Након тога, преко DMA модула прослеђују се 8-битни пиксели слике сваког такта нашем акцелератору. Акцелератор врши хардверско филтрирање и обрађену слику уписује у RAM бафер из ког можемо потом прочитати слику.

Реализовани систем састоји се од:

- Zynq процесорског система (dual-core Cortex A9)
- AXI Timer модула за мерење перформанси
- AXI DMA модула
- Хардверског акцелератора за филтрирање слике (acc_image_filters)
- Додатних модула за повезивање периферија и слејв компоненти



Слика 1. Блок дијаграм комплетног система из Vivado Block Design-а



Слика 2. Блок дијаграм система

Акцелератор би на почетку, у интерно имплементиране *BRAM* блокова учитао део слике, како би након тога сваки такт могао да исписује излазне вредности. Имплементирани су 16-битни *BRAM* блокови који би својом дужином имали довољно меморијских локација за 2 реда слике, односно сасвим довољно вредности за почетак рада уколико би кернел био величине 3x3. За сваку већу величину кернела додавао би се још један BRAM блок, до максималних 4 BRAM блокова. Бонус поени били би остварени уколико би имплементирали BRAM блок са већом ширином речи и с тиме смањили број потребних BRAM блокова

Константним читањем BRAM блокова и надолазећих пиксела, па потом множењем са коефицијентима филтера из регистара добијамо излазни пиксел нашег акцелератора.

2.1. Архитектура хардверског акцелератора

Акцелератор(acc_image_filter) представља модификовани *AXI Lite* модул, односно садржи у себи машину стања коју би садржао *AXI Lite* и подржава његов вид комуникације. Састоји се од 4 главне подкомпоненте:

- BRAM меморијски блокови (BRAM)
- Јединица за обраду шиксела (axi_image_block)
- Регистарске компоненте са *AXI Lite* машином стања (axi_registers)

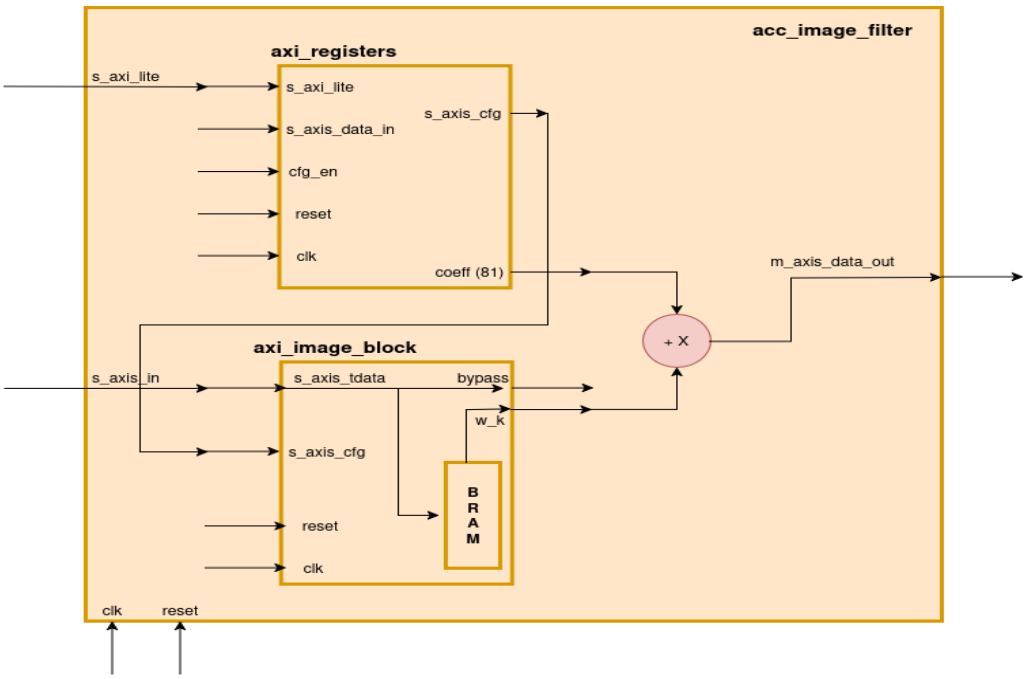
Интерфејс акцелератора приказан је у Табели 1:

Име сигнала / интерфејса	Путања	Опис
clk	in	Сигнал такта
reset	in	Синхрони ресет активан у логичкој јединици
s_axi_lite_cfg	in	AXI Lite интерфејс за приступ конфигурационим регистрима
s_axis_data_in	in	AXI Lite интерфејс за пријем улазних пиксела
m_axis_data_out	out	AXI Lite интерфејс за слање необрађених пиксела

Табела 1. Интерфејси хардверског акцелератора

Акцелератор функционише по следећим корацима:

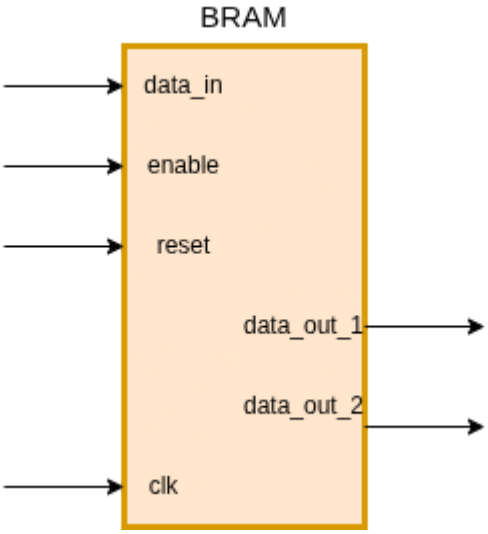
1. Унос конфигурационих регистара
2. Унос довољног броја редова у BRAM блокове док се не напуне
3. Множење коефицијената филтера са коефицијентима прозора, најплајновано са закашњењем од 6-9 тактних периода



Слика 3. Блок дијаграм хардверског акцелератора (acc_image_filter)

2.2.1. BRAM меморијски подсистем

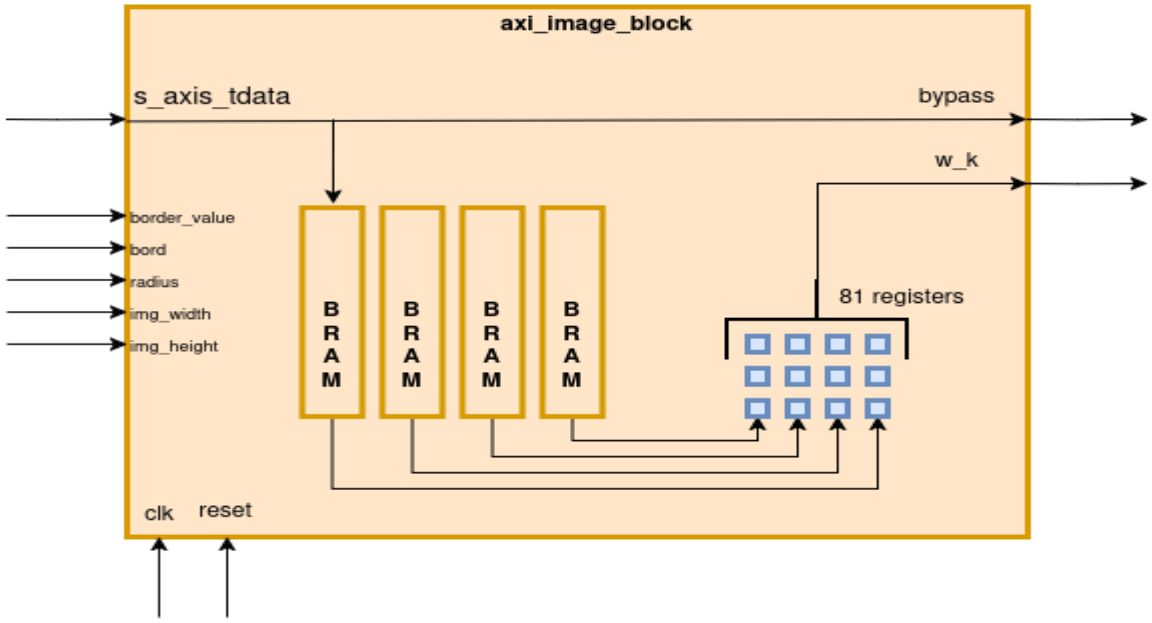
За складиштење слике ради брже обраде пиксела користе се BRAM блокови. Реализовани блокови подржавају упис вредности пиксела на улаз која би сваки такт била пропагирана дубље у блок. Сваки такт би вредности уписане на последњој меморијској локацији биле исписане.



Слика 4. Структура BRAM компоненте

2.2.2. Јединица за обраду пиксела

Јединица за обраду пиксела је подкомпонента која води рачуна о величини филтера, слике и појединих конфигурационих параметара, и на свој излазни интерфејс остатку акцелератора пружа пикселе које треба обрадити. Унутар себе компонента за обраду пиксела имплементира BRAM блокове, помоћу којих успева да постигне рачунање сваки такт.



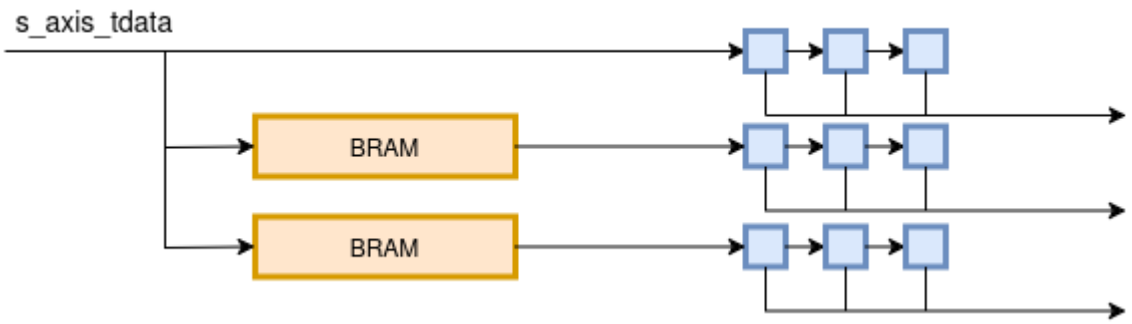
Слика 5. Структура axi_image_block компоненте

Интерфејс јединице за обраду пиксела приказан је у Табели 2:

Име сигнала / интерфејса	Путања	Опис
clk	in	Сигнал такта
reset	in	Синхрони ресет активан у логичкој јединици
border_value	in	Регистар вредности ивице
bord	in	Регистар мода обраде ивице
radius	in	Регистар величине филтера
img_width	in	Регистар ширине слике
img_height	in	Регистар висине слике
ready	in	Сигнал спремности система да прими податке, прослеђено од AXI lite
s_axis	in	AXI Lite интерфејс за пријем пиксела
bypass	out	AXI Lite интерфејс за слање необрађених пиксела
w_k	out	AXI Lite интерфејс за слање обрађених пиксела

Табела 2. Интерфејси јединице за обраду пиксела

Јединица врши “бирање” пиксела за обраду тако што узме излазну вредност из BRAM блока, као и надолазећи пиксел, и упише их у *w_k* регистре. Такве вредности пропагира осталим регистрима као на Слици 6.

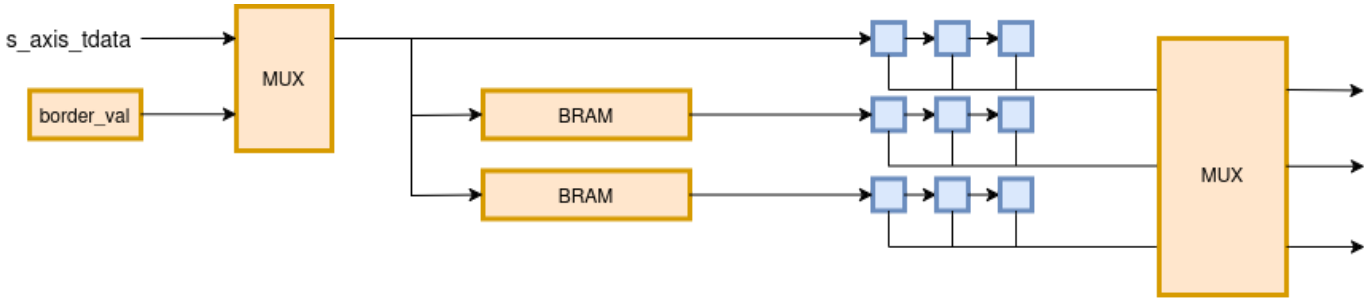


Слика 6. Реализација филтера

Вредности контролног регистра тумачимо као 16-битну бинарну вредност конфигурационих параметара. Парсирањем контролног регистра добијамо засебне конфигурационе регистре чије информације прослеђујемо јединици за обраду пиксела. Променом контролног регистра можемо изабрати различите величине филтера. Веће величине изискују више BRAM блокова и прихватних регистара. За radius = n имамо n број BRAM блокова и 2n+1 величину филтера, односно толико прихватних регистара.

Конфигурациони регистар bord нам говори како ћемо обрађивати ивице. У процесу филтрирања, уколико одлучимо да акцелератор не обрађује ивице улазне слике јер за њих нема све вредности прозора, на ивицама излазне слике исписујемо 0, односно нема ивице.

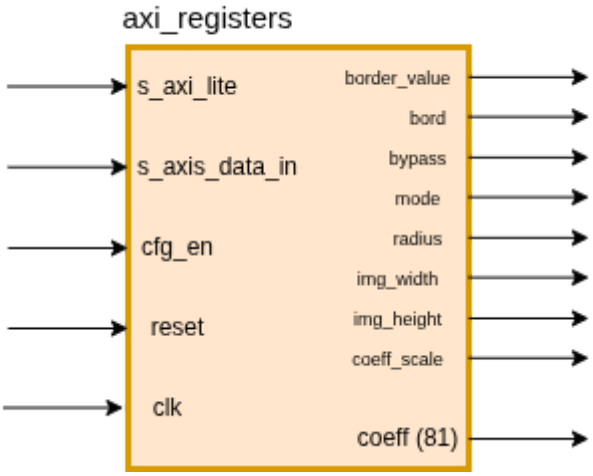
У случају да изаберемо мод проширивања ивице, слику привидно проширимо константним вредностима уписаних у регистар *border_value*. Ово реализујемо тако што не пунимо BRAM блокове на почетку и крају са вредностима које нам долазе, већ са вредностима из *border_value* регистра. Леве и десне ивице, обрађујемо са мултиплексерима који се налазе на улазима у *w_k* регистре. Овај процес је сликовито приказан на Слици 7.



Слика 7. Реализација мода проширења константним вредностима

2.2.3. Регистарска компонента

Регистарски интерфејс (axi_registers) омогућава процесору конфигурацију рада акцелератора. Регистри су меморијски мапирани и приступа им се преко *AXI-Lite* магистрале.



Слика 8. Блок дијаграм axi_registers компоненте

Регистри које ове компонента садржи су:

- reg_ctrl - контролни регистар
- reg_img_w - ширина слике
- reg_img_h - висина слике
- reg_coeff_scale - фактор скалирања коефицијента у Q4.7 формату
- reg_coeff_W0 до reg_coeff_W80 - коефицијенти од W0 до W80 филтра у Q1.15 формату

Контролни регистар REG_CTRL (Табела 3) садржи контролне битове за:

- MODE
- BYPASS
- BORD
- BORDER_VALUE

Поље MODE дефинише која вредност се шаље на излаз модула:

0: Излазна вредност је неозначени 8-битни податак.

1: Излазна вредност је означени 16-битни податак при чему се 9 бита користи за цео део, а 7 бита за разломљени део(Q9.7).

Поље BYPASS одређује излаз из акцелератора:

0: На излаз се прослеђује вредност дефинисана MODE битовима.

1: На излаз се прослеђују вредности улазних пиксела.

Поље BORD дефинише вредност којом се допуњује слика:

0x: Не обрађују се елементи који немају потпуно локално суседство.

10: Елементи који излазе ван оквира слике имају вредност најближег елемента који се налази у слици.

11: Елементи који излазе ван оквира слике имају вредност дефинисану BORDER_VALUE вредношћу.

Поље BORDER_VALUE дефинише 8-битну вредност која се користи за елементе локалног суседства који излазе ван оквира слике у случају када опција проширења BORD има вредност 11.

not used				border_value								bord		bypass	mode
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Табела 3. Регистар reg_ctrl, Offset = 0x0

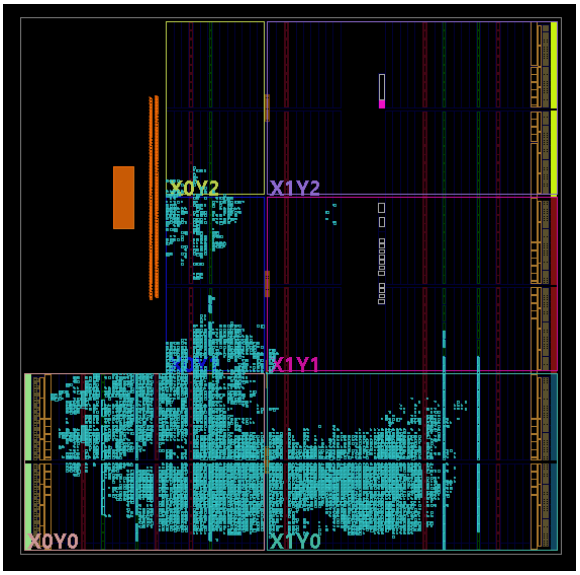
2.2. Синтеза и имплементација система

Током имплементације, у нашој реализацији има јако мали проблем са тајмингом. Понекад се појави, понекад не и мислим да је то до како компајлер у окружењу оптимизује код.

LUT	FF	BRAM	URAM	DSP
9,016	11,281	2	0	98

Табела 4. Заузеће ресурса на Pynq Z2 развојној плочици

На следећој слици приказано је како *Vivado* имплементира дизајн на наш чип.



Слика 9. Screenshot из Vivado Implementation Design

3. Реализација софтверског дела система

Софтверски део пројекта имплементира контролу, конфигурацију и тестирања акцелератора за убрзање филтрирања слике. Софтвер има следеће задатке:

- Иницијализација меморије
- Упис слике у Data buffer
- Конфигурација регистара акцелератора
- Покретање обраде
- Чекање завршетка
- Провера резултата
- Читање резултата из Result buffer-a
- Мерење времена извршавања

За тестирање је коришћена debug конзола и команде:

- mwr - за упис слике у Data buffer
- mrd - за читање резултата из Result buffer-a

Тиме је омогућено ручно тестирање акцелератора без потребе за додатним софтверским wrapper-ом.

3.1. Структура

Главна структура параметара која описује режим рада алгоритма зове се ProcessingParams и садржи следећа поља:

Поље	Тип	Опис
ImgW	*u16*	Ширина слике (број колона у пикселима).
ImgH	*u16*	Висина слике (број редова у пикселима).
FilterRadius	*u8*	Полупречник филтрационог језгра, користи се као Radius и одређује kernel величине (2*Radius+1).
FilterCoeff	*s16**	Показивач на низ коефицијената филтера. У коду је то уграђени низ од 81 елемената (s16 FilterCoeffs[81]), а не показивач.
FilterCoeffScale	*u16*	Фактор скалирања резултата филтрирања. Користимо га у HW конфигурацији (AccConfigure).
Border	*u8*	Начин третирања ивица слике. У коду је enum Border са вредностима: NO_PAD, CONST_PAD, NEAREST_PAD.
Bypass	*u8*	Омогућава прескакање обраде. У коду је enum Bypass са вредностима: YES_BYPASS, NO_BYPASS.
ModeType	*enum MODE*	Дефинише излазни формат (bit8 или bit16).
BorderValue	*u8*	Вредност константе за ивице када је Border = CONST_PAD

Табела 5. Табела структуре ProcessingParams

Структура ***ProcessingParams*** омогућава флексибилну конфигурацију система у run-time режиму рада. Параметри се користе како у софтверској, тако и у хардверској имплементацији филтрирања, чиме се обезбеђује идентично понашање оба система током тестирања и поређења перформанси.

Поље ***ImgW*** і ***ImgH*** дефинишу димензије слике која се обрађује. Ове вредности одређују број итерација у петљама софтверске имплементације, као и границе адресирања меморије у хардверском акцелератору.

Поље ***FilterRadius (Radius)*** одређује величину филтрационог језгра. Величина маске рачуна се као (2*Radius + 1) x (2*Radius + 1). На пример, за вредност 1 добија се 3x3 филтер, док вредност 2 одговара 5x5 филтеру.

Поље ***FilterCoeffs*** представља уграђени низ од 81 целобројних коефицијената филтера. Коефицијенти су смештени у једнодимензионалном низу, редоследом по врстама (row-major order). Укупан број елемената који се користи зависи од радијуса филтера.

Поље ***FilterCoeffScale*** користи се за нормализацију резултата филтрирања. Након израчунавања суме тежинских производа суседних пиксела и коефицијената, резултат се дели овом вредношћу како би се добила коректна амплитуда излазног пиксела у опсегу од 0 до 255.

Поље ***BorderType (Border)*** дефинише начин третирања ивица слике. Софтверска имплементација

подржава три режима:

- ***NO_PAD*** - пиксели ван слике се игноришу, резултат се поставља на 0.
- ***CONST_PAD*** - пиксели ван слике добијају константну вредност дефинисану у пољу **BorderValue**.
- ***NEAREST_PAD*** - пиксели ван слике добијају вредност најближег граничног пиксела.

Поље ***BypassType (Bypass)*** омогућава да се акцелератор користи без извршавања филтрирања. Ако је постављено на ***YES_BYPASS***, улазни пиксели се директно прослеђују на излаз без обраде (за 8-битни излаз) или се конвертују у Q9.7 формат (за 16-битни излаз). Ако је вредност ***NO_BYPASS***, примењује се задати филтер.

Поље ***ModeType*** одређује излазни формат резултата:

- bit8 - сатурација резултата у опсегу од 0 до 255.
- bit16 - Q9.7 фиксна тачка, сатурација у опсегу од -32768 до 32767.

На овај начин, структура ***ProcessingParams*** обезбеђује да софтверска и хардверска имплементација користе исте параметре како би дале идентичне резултате, што је кључно за верификацију акцелератора и поређење перформанси.

3.2. Опис функција

1. ***ImageFilterSW***

```
static void ImageFilterSW(u8* DataBuffer, u8* ResultBuffer, ProcessingParams Params)
```

Опис функције:

Функција имплементира софтверску верзију алгоритма за филтрирање слике. Обрада функције се извршава секвенцијално на процесорском систему Zynq (Cortex-A9 језгра на PYNQ-Z2 плочи) процесору проласком кроз сваки пиксел слике и применом задатог филтрационог језгра. Резултат филтрирања уписује се у излазни бафер.

Улазни параметри:

- *u8** ***DataBuffer*** - показивач на улазни низ пиксела (слика у меморији)
- *u8** ***ResultBuffer*** - показивач на низ где се чувају резултати обраде
- ***ProcessingParams Params*** - структура са параметрима филтрирања

Опис рада:

Функција пролази кроз слику помоћу две угњеждене петље - по висини (редови) и ширини (колоне). Ако је параметар ***BypassType*** постављен на ***YES_BYPASS,*** филтрирање се прескаче. У том случају, за 8-битни излаз пиксели се директно копирају у излазни бафер, док се за 16-битни излаз конвертују у Q9.7 формат померањем за 7 бита. У супротном, за сваки пиксел се формира локално суседство димензије (2**Radius*+1) x (2**Radius*+1).

За сваки елемент суседства врши се:

- Множење вредности пиксела одговарајућим коефицијентом филтера
- Сабирање свих тежинских производа

Након акумулације, добијена сума се дели са ***FilterCoeffScale*** ради нормализације резултата.

Добијена вредност се затим сатурише у складу са излазним форматом:

- bit8 - вредност се ограничава на опсег од 0 до 255
- bit16 - вредност се конвертује у Q9.7 фиксну тачку где се вредност помера се за 7 бита и онда се бира тип сатурације:
  - За **LOG7** і **SHARP_BOX9** филтере користи се **unsigned** сатурација (од 0 до 65535), јер коефицијенти садрже негативне вредности и потребно је померање у позитивни опсег.
  - За остале филтере користи се **signed** сатурација (од - 32768 до 32767), јер коефицијенти имају само позитивне вредности.

Када се филтрирање примењује на пикселе који се налазе на ивици слике, начин обраде одређује се параметрима ***BorderType:***

- ***NO_PAD*** - пиксели ван слике се игноришу, резултат се поставља на 0
- ***CONST_PAD*** - пиксели ван слике добијају константну вредност дефинисану у пољу **BorderValue**
- ***NEAREST_PAD*** - пиксели ван слике добијају вредност најближег граничног пиксела

Софтверска имплементација служи као референтна верзија за проверу тачности хардверског акцелератора и за поређење перформанси.

2. ***ImageFilterHW***

```
static int ImageFilterHW(u8* DataBuffer, u8* ResultBuffer, ProcessingParams Params)
```

Опис функције:

Функција управља хардверском акцелерацијом филтрирања слике коришћењем наменски пројектованог IP језгра на ***Zynq PYNQ-Z2*** плочи. Она врши конфигурацију акцелератора, покретање обраде и чекање завршетка операције. Резултат филтрирања уписује се у излазни бафер.

Улазни параметри:

- *u8** ***DataBuffer*** - показивач на улазни низ пиксела (слика у меморији)
- *u8** ***ResultBuffer*** -показивач на низ где се чувају резултати обраде(хардверски резултати)
- ***ProcessingParams Params***- структура са параметрима филтрирања

Опис рада:

Функција најпре уписује параметре филтрирања у меморијски мапиране регистре акцелератора. То укључује:

- Димензије слике (***ImgW***, ***ImgH***)
- Параметре филтера (***Radius***, ***FilterCoeffsScale***)
- Контролне битове (***BorderType***, ***BypassType***, ***ModeType***, ***BorderValue***)
- Коефицијенте филтера (до 81 вредности у зависности од радијуса)

Након конфигурације, функција покреће DMA трансфер:

- TX канал шаље улазне податке из *DataBuffer* ка акцелератору
- RX канал прима обрађене податке из акцелератора у *ResultBuffer*

Током преноса користе се прекидне рутине које постављају flag-ови (*TxDone*, *RxDone*) када је пренос завршен. Функција затим чека да оба преноса буду комплетирана или да истекне дефинисани *timeout*. По завршетку, резултати су доступни у меморији и могу се даље користити или поредити са софтверским резултатом помоћу функције *CheckData()*.

Хардверска имплементација омогућава значајно убрзање захваљујући паралелизму и pipelining архитектури, чиме се постиже обрада једног пиксела по такту након иницијалног кашњења.

3. ***CheckData***

```
static int CheckData(u8* ResultBuffer, u8* ReferentBuffer, ImageShape Img);
```

Опис функције:

Функција проверава исправност резултата поређењем хардверски обрађене слике са референтним (софтверским) резултатом. Користи се за валидацију акцелератора и потврду да хардверска имплементација даје идентичне резултате као софтверска

Улазни параметри:

- *u8** ***ResultBuffer*** -показивач на низ где се чувају резултати обраде (хардверски резултати)
- *u8** ***ReferentBuffer*** - показивач на низ где се чувају резултати обраде (софтверски резултати)
- ***ImageShape Img*** - структура која садржи димензије слике ***(ImgW, ImgH)***

Опис рада:

Функција најпре инвалидира кеш меморију излазног бафера помоћу ***Xil_DCacheInvalidateRange()*** како би се учитали најновији подаци из DDR меморије које је уписао DMA контролер.

Затим пролази кроз целу слику помоћу две угњеждене петље - по висини (редови) и ширини (колоне). За сваки пиксел пореди вредности у ***ResultBuffer*** и ***ReferentBuffer***. Ако постоји разлика, исписује поруку о грешци са редом, колоном и вредностима (добијени резултат и очекивани резултат).

Грешке се броје помоћу бројача, а на крају се исписује укупан број пронађених грешака. Функција враћа **XST_SUCCESS** чак и ако постоје разлике, што омогућава да се тестирање настави и да се види комплетна статистика одступања.

Софтверска функција ***CheckData()*** служи као механизам верификације и потврде да хардверски акцелератор ради исправно и даје резултате идентичне софтверској имплементацији.

4. ***DmaConfigure***

```
static int DmaConfigure(XAxiDma_Config* AxiDmaConfigPtr, XAxiDma* AxiDmaPtr);
```

Функција иницијализуј DMA контролер помоћу ***XAxiDma_CfgInitialize()***. Проверава да ли је DMA у scatter-gather режиму (мора бити у simple transfer режиму). Затим конфигурише прекиде за TX и RX канале помоћу ***XSetupInterruptSystem()***, уз дефинисану приоритетну вредност. На крају омогућава све прекиде за оба канала помоћу ***XAxiDma_IntrEnable()***. Ако је конфигурација успешна, враћа **XST_SUCCESS**, у супротном пријави ће грешку.

5. ***DmaStartTransfers***

```
static int DmaStartTransfers(XAxiDma* AxiDmaPtr, u8* TxBuffer, u32 TxSize, u8* RxBuffer, u32 RxSize);
```

Пре покретања преноса, функција освежава кеш меморију помоћу ***Xil_DCacheFlushRange()*** ако би се обезбедила синхронизација између кеша и DDR меморије. Затим се покреће DMA трансфер за пријем података (RX канал) помоћу ***XAxiDma_SimpleTransfer()***, а затим и DMA трансфер за слање података (TX канал). Ако су оба преноса успешно покренута, враћа **XST_SUCCESS**, у супротном пријављује грешку.

6. ***DmaWaitTransfers***

```
static int DmaWaitTransfers(volatile u32* TxFlag, volatile u32* RxFlag, u32 Timeout);
```

Функција користи ***Xil_WaitForEventSet()*** где чека да се постави TX fleg (***TxDone***). Ако TX пренос не заврши у дефинисаном времену, пријављује грешку. Након чега чека да се постави RX flag (***RxDone***). Ако RX пренос не заврши у дефинисаном времену, пријављује грешку. Ако оба преноса заврше успешно, функција враћа **XST_SUCCESS**.

7. ***AccConfigure***

```
static int AccConfigure(UINTPTR BaseAddress, ProcessingParams Params)
```

Функција формира вредност контролног регистра (***regCtrlVal***) комбиновањем више параметара:

- ***ModeType*** - дефинише излазни формат (8-bit или 16-bit Q9.7)
- ***BorderType*** - начин третирања ивица слике (**NO_PAD**, **CONST_PAD**, **NEAREST_PAD**)
- ***BypassType*** - да ли се акцелератор прескаче или користи
- ***BorderValue*** - вредност која се користи за проширење ивица у **CONST_PAD** режиму

Након тога се у регистре акцелератора уписују основни параметри:

- Радијус филтера (***Radius***)
- Фактор скалирања коефицијената (***FilterCoeffsScale***)
- Димензије слике (***ImgW***, ***ImgH***)

Затим се у акцелератор уписује свих 81 коефицијената филтра, распоређених у једнодимензионалном низу. Сваки коефицијент се уписује у посебан регистар померен за 4 бајта у односу на претходни.

Ако је конфигурација успешно завршена, функција враћа **XST_SUCCESS**. На овај начин акцелератор добија све потребне параметре да би могао да изврши филтрирање слике у складу са задатим режимом рада.

8. ***TxIntrHandler i RxIntrHandler***

```
static void TxIntrHandler(void *Callback);
static void RxIntrHandler(void *Callback);
```

Функције најпре читају статус прекида за TX канал помоћу **XAxiDma_IntrGetIrq()**. Након тога потврђују (acknowledge) да је прекид обрађен помоћу **XAxiDma_IntrAckIrq()**, чиме се спречавају понављање истог прекида. Ако је постављен бит ***Interrupt On Complete (IOC)***, значи да је пренос података завршен. У том случају се поставља flag ***TxDone = 1***, што остатак програма користи као сигнал да је TX пренос успешно завршен. Исти кораци се пролазе и код преноса RX.

3.3. Код функције *ImageFilterSW*

У следећем делу текста приказан и описан скелетом софтверског дела кода ***ImageFilterSW*** шта сваки сегмент представља и како доприноси целокупној функционалности система за филтрирање слике. Циљ је да се јасно покаже логика имплементације, начин на који се параметри користе и како се обезбеђује идентично

понашање софтверске и хардверске верзије алгоритма.

На самом почетку функције дефинисани су параметри променљиве које описују основне карактеристике слике и филтера:

- **width** и **height** - димензије слике (ширина и висина у пикселима), узете из структуре ***Params.Img.***
- **radius** - полупречник филтера, одређује величину кернела.
- **kernelSize** - димензија кернела, рачуна се као kao 2 * radius + 1.
- **scale** - фактор скалирања коефицијената филтера, користи се за нормализацију резултата.

Овај део кода поставља основу за даље рачунање, јер се сви наредни кораци филтрирања ослањају на ове вредности.

```
int x, y, i, j;

// Dimenzije slike
const int width  = Params.Img.ImgW;
const int height = Params.Img.ImgH;

// Radijus filtera i velicina kernela (2*radius+1)
const int radius = Params.Radius;
const int kernelSize = 2 * radius + 1;

// Faktor skaliranja koeficijenata
const int scale = Params.FilterCoeffsScale;
```

Слика 10. Иницијализација основних параметара

Следећи део кода проверава се да ли је укључен **Bypass** режим (**Params.BypassType=YES_BYPASS**) и ако је **BYPASS** укључен, филтрирање се прескаче при чему се за bit8 излаз пиксели се директно копирају из улазног бафера у излазни, а за bit16 излаз пиксели се конвертују у Q9.7 формат померањем за 7 бита. Након овога функција се одмах завршава (return).

Овај део омогућава да се акцелератор или софтверска функција користи и у режиму када није потребно филтрирање, већ само прослеђивање података.

```
// Provera BYPASS moda (mora da se poklapa sa hardverskom implementacijom!)
if (Params.BypassType == YES_BYPASS)
{
    int total = width * height;

    if (Params.ModeType == bit8)
    {
        // Ako je izlaz 8-bitni, samo se kopira svaki piksel
        for (int k = 0; k < total; k++)
            ResultBuffer[k] = DataBuffer[k];
    }
    else
    {
        // Ako je izlaz 16-bitni, podaci se konvertuju u Q9.7 format
        int16_t* out = (int16_t*)ResultBuffer;
        for (int k = 0; k < total; k++)
            out[k] = ((int16_t)DataBuffer[k]) << 7; // pomeraj za fiksnu tacku, Q9.7 format
    }

    return;
}
```

Слика 11. Провера BYPASS мода

У наредном делу кода приказана је **главна петља** која пролази кроз сваки пиксел слике. За сваки пиксел се формира локално суседство димензија (2*radius+1)x(2*radius+1) где се вредности суседних пиксела множе са одговарајућим коефицијентима филтера и сабирају у променљиву **sum**. Ако пиксел нема комплетно суседство (у случају ***NO_PAD*** режима), означава се као невалидан и резултат се поставља на 0. Овај део кода представља основу алгоритма за секвенцијално филтрирање слике пиксел по пиксел.

А на слици десно приказано је **третирање ивица слике (Border handling)**. Када пиксели излазе ван граница слике.

На крају се рачуна индекс коефицијента у кернелу помоћу израза (j + radius) * kernelSize + (i + radius), и вредност пиксела се множи са одговарајућим коефицијентом филтера, а резултат се додаје у акумулацију (**sum**).

```
// Glavna petlja - prolazak kroz svaki piksel slike
// Za svaki piksel se uzima lokalno susedstvo dimenzija (2*radius+1)x(2*radius+1),
// racuna se tezinska suma i upisuje rezultat u izlazni bafer.

for (y = 0; y < height; y++)
{
    for (x = 0; x < width; x++)
    {
        int32_t sum = 0;   // akumulacija rezultata
        int invalid = 0;   // indikator da li piksel ima nepotpuno susedstvo (NO_PAD slucaj)

        // Prolazak kroz kernel
        for (j = -radius; j <= radius; j++)
        {
            for (i = -radius; i <= radius; i++)
            {
                int yy = y + j;
                int xx = x + i;
                u8 pixel = 0;

                //BORDER HANDLING (Tretiranje ivica slike)
                if (Params.BorderType == NO_PAD)
                {
                    // Ako piksel izlazi van slike ==> oznaci ga kao nevalidan
                    if (yy < 0 || yy >= height || xx < 0 || xx >= width)
                    {
                        invalid = 1;
                        break;
                    }
                    pixel = DataBuffer[yy * width + xx];
                }
                else if (Params.BorderType == CONST_PAD)
                {
                    // Ako piksel izlazi van slike ==> koristi se konstanta
                    if (yy < 0 || yy >= height || xx < 0 || xx >= width)
                    {
                        pixel = Params.BorderValue;
                    }
                    else
                    {
                        pixel = DataBuffer[yy * width + xx];
                    }
                }
                else // NEAREST_PAD (ponavljanje granicnih piksela - BORD = 10)
                {
                    int yyy = yy;
                    int xxx = xx;

                    if (yyy < 0) yyy = 0;
                    if (yyy >= height) yyy = height - 1;
                    if (xxx < 0) xxx = 0;
                    if (xxx >= width) xxx = width - 1;

                    pixel = DataBuffer[yyy * width + xxx];
                }

                // Racunanje indeksa koeficijenta u kernelu
                int coeffIndex = (j + radius) * kernelSize + (i + radius);
                sum += pixel * Params.FilterCoeffs[coeffIndex];
            }
        }

        if (invalid) break; // prekid ako piksel nema validno susedstvo
    }
```

Слика 12. Главна петља и третирање ивица

И на крају последњи део кода приказује завршну фазу обраде сваког пиксела. Најпре се обрађује случај када пиксел нема комплетно суседство (**NO_PAD** режим). У том случају излазна вредност се поставља на 0, било да је излаз у 8-битном или 16-битном формату.

Затим се врши скалирање резултата где се акумулисана сума се дели са фактором скалирања (**FilterCoeffsScale**) како би се нормализовала вредност.

На крају се резултат уписује у излазни бафер, уз сатурацију у зависности од излазног формата.

И овим делом кода обезбедили смо да излазни пиксели увек буду у валидном опсегу, да се резултати правилно ограничавају у зависности од типа филтера и да се резултати софтверске имплементације поклапају са хардверском, чиме се постиже конзистентност система.

```
// NO_PAD ponasanje
if (invalid)
{
    // Ako piksel nema kompletno susedstvo ==> izlazna vrednost je 0
    if (Params.ModeType == bit8)
    {
        ResultBuffer[y * width + x] = 0;
    }
    else
    {
        ((int16_t*)ResultBuffer)[y * width + x] = 0;
    }
    continue;
}

// Skaliranje rezultata
if (scale != 0)
    sum /= scale;

// Upis rezultata u izlazni bafer
if (Params.ModeType == bit8)
{
    // Ogranicavanje rezultata na 8-bitni opseg (0-255)
    if (sum < 0) sum = 0;
    if (sum > 255) sum = 255;

    ResultBuffer[y * width + x] = (u8)sum;
}
else
{
    // Q9.7 fiksna tacka (16-bitni format)
    int32_t val = sum << 7;

    // Provera da li je aktivan LOG7 ili SHARP filter
    if (memcmp(Params.FilterCoeffs, LOG7, sizeof(LOG7)) == 0 || memcmp(Params.FilterCoeffs, SHARP_BOX9, sizeof(SHARP_BOX9)) == 0)
    {
        // LOG7 i SHARP koriste unsigned saturaciju
        // Ogranicavanje rezultata na opseg (0 do 65535)
        val += 32768;
        if (val < 0) val = 0;
        if (val > 65535) val = 65535;
        ((int16_t*)ResultBuffer)[y * width + x] = (int16_t)val;
    }
    else
    {
        // Ostali filteri koriste signed saturaciju
        // Ogranicavanje rezultata na 16-bitni opseg (-32768 do 32767)
        if (val < -32768) val = -32768;
        if (val > 32767)  val = 32767;
        ((int16_t*)ResultBuffer)[y * width + x] = (int16_t)val;
    }

}
```

Слика 13. NO_PAD понашање, скалирање и упис резултата

4. Резултати и мерења

За мерење времена извршавања софтверске и хардверске детекције користи се AXI Timer периферија. Тајмер се иницијализује, ресетује и покреће пре почетка сваке обраде. По завршетку обраде, тајмер се зауставља и чита број тактова (SwProcessingTime и HwProcessingTime). За пример слику користили смо класичну Lena 128x128 слику која се обично и користи.

Формула за израчунавање убрзања је: *Speedup = SwProcessingTime / HwProcessingTime*

За проверу исправности рада акцелератора и програма, користили смо конфигурацију:

Border Type = CONST_PAD
Border Value = 128

И филтере:

3x3 BOX са 8-битним излазом
5x5 Gauss са 8-битним излазом
7x7 LOGса 16-битним излазом
9x9 BOX са 16-битним излазом
9x9 BOX bypass са 8-битним излазом
3x3 BOX bypass са 16-битним излазом
9x9 BOX без проширења са 8-битним излазом

Резултати софтверског филтрирања дати су на сликама 14.

14.a) бит8 3x3 BOX

14.b) бит8 5x5 Gauss

14.c) бит16 7x7 LOG

14.d) бит16 9x9 BOX

14.e) бит16 3x3 bypass

Софтверска реализација је углавном тачна. Имали смо проблем са no_pad реализацијом. Остале слике филтера које се не налазе у оквиру слика 14. случано нису сачуване, али верујте нам на реч.

Резултати хардверског филтрарања дати су на сликама 15.

15.a) бит8 3x3 BOX

15.b) бит8 5x5 Gauss

15.c) бит16 7x7 LOG

15.d) бит16 9x9 BOX

15.e) бит8 9x9 BOX bypass

15.f) бит16 3x3 BOX bypass

15.g) бит16 9x9 BOX nopad

По сликама можемо да видимо да раде сва проширења и величине филтера. Bypass мод има својих проблема и није у потпуности тачан. Што се тиче филтера попут LOG и Sharp који имају негативне коефицијенте, акцелератор не израчунава пикселе како треба, односно долази до прекорачења у излазним регистрима. То можемо закључити зато што контуре са слике описују облик Лене.

Заједно са сликама измерена је и брзина и софтверске и хардверске акцелерације. Резултати су приказани у табели 6.

Филтер	Величина	Излазни мод	Bypass	Проширење	SW [такт]	HW [такт]	Убрзање
BOX	3x3	8 бит	не	да	11397234	321108	35.49
Gauss	5x5	8 бит	не	да	1721160	321121	5.35
LOG	7x7	16 бит	не	да	7903954	326199	24.23
BOX	9x9	16 бит	не	да	7931365	321102	24.41
BOX	9x9	8 бит	да	да	12745191	326205	39.07
BOX	3x3	8 бит	да	да	12803490	321107	39.87
BOX	9x9	8 бит	не	не	37427	321098	0.11

Табела 6. Време извршавања за различите димензије слике

Из ове табеле можемо уочити да је убрзање знатно веће. Неки филтери раде горе или боље од других, али мислим да је то због грешака које имамо у хардверском коду. Софтверско решење када има проширења нема смисла уопште. Када се упали bypass требало би да се смањи време процесирања, додуше резултати не говоре тако.

5. Закључак

Овим пројектом савладали смо техникре програмирања FPGA чипова као и прављења функционалних дигиталних система. Заједно са хардвером, софтвер заокружује целу причу и функционалност система.

По захтеву пројекта, требало је имплементирати могућност филтрирања секвенционално, слику по слику, што нисмо стигли да убацимо. Такође је требало реализовати филтрирање и слике 512x512 без поновног синтетисања хардвера што такође нисмо реализовали. Још једна ствар коју нисмо направили је провера или забрана уписа у конфигурационе регистре током рада акцелератора. Све ове ствари верујем да није тешко убацити али због обимности пројекта и недостатка времена били смо онемогућени.